

Verifiable and Pairing-Free Certificateless Searchable Encryption with Complete Keyword-Guessing Attack Resistance

Mohammed Wasim Khan

Abstract—The rapid proliferation of Mobile and Medical Internet of Things (MIoT) has driven the adoption of cloud servers for outsourced data storage. However, preserving data privacy while maintaining searchability remains a critical challenge. Certificateless Searchable Encryption (CL-SE) eliminates the certificate management overhead of traditional PKI and the key escrow problem of Identity-Based Cryptography. Yet, existing CL-SE schemes suffer from three major limitations: (1) heavy reliance on computationally expensive bilinear pairings; (2) vulnerability to Outside Keyword-Guessing Attacks (OKGA) when trapdoors are intercepted; and (3) a lack of verifiable search mechanisms, forcing users to blindly trust that the “honest-but-curious” cloud server has returned complete results. In this paper, we propose a novel pairing-free, verifiable certificateless searchable encryption scheme designed for IoT-cloud environments. The proposed scheme achieves simultaneous resistance against both Inside and Outside Keyword-Guessing Attacks (IKGA/OKGA) through a server-bound trapdoor blinding mechanism. Furthermore, we integrate a Merkle Hash Tree (MHT) structure to guarantee search completeness and soundness. Formal security proofs in the random oracle model demonstrate that the scheme’s security reduces to the hard Computational Diffie-Hellman (CDH) problem. Theoretical performance analysis and experimental simulations confirm that the proposed pairing-free construction is highly efficient and scalable, making it ideally suited for deployment in resource-constrained MIoT networks.

Index Terms—Searchable Encryption, Certificateless Cryptography, Keyword Guessing Attacks, Verifiable Search, Internet of Things, Merkle Hash Tree

1 INTRODUCTION

THE integration of Internet of Things (IoT) devices with cloud computing paradigms has revolutionized data processing, particularly in domains such as electronic healthcare and smart cities. Because IoT devices possess limited storage and computational capacity, outsourcing encrypted data to external cloud servers has become standard practice. However, outsourcing introduces severe privacy concerns. To retrieve specific files without decrypting the entire database, Searchable Encryption (SE) was introduced, allowing a cloud server to test encrypted keywords against encrypted trapdoors.

While early Public Key Encryption with Keyword Search (PEKS) schemes relied on traditional Public Key Infrastructure (PKI), the burden of managing and revoking digital certificates is prohibitively expensive for IoT networks. Identity-Based SE (IB-SE) resolved certificate management but introduced the key escrow problem, where a central Private Key Generator knows all user keys. Certificateless Searchable Encryption (CL-SE) solves both issues by splitting the private key between a Key Generation Center (KGC) and the user.

1.1 Research Gap and Problem Statement

Despite the architectural advantages of CL-SE, deploying current schemes in dynamic IoT environments presents

three unsolved challenges, which represent the core research gap addressed in this paper:

1.1.1 The Computational Bottleneck of Pairings

The vast majority of existing CL-SE schemes [1], [2] rely heavily on bilinear pairing operations over elliptic curves to establish shared secrets during the search phase. Bilinear pairings are notoriously resource-intensive, requiring significantly more clock cycles than standard scalar multiplication. In resource-constrained environments like Mobile IoT (MIoT), this computational overhead results in unacceptable latency and severe battery drain.

1.1.2 The Keyword Guessing Attack (KGA) Vulnerability

Because the space of searchable keywords is inherently low-entropy (e.g., dictionary words, medical terms), SE schemes are highly susceptible to offline guessing attacks. If an adversary captures a trapdoor, they can iteratively encrypt guesses from a dictionary and test them against the trapdoor until a match is found. While recent schemes [2] attempt to protect against a malicious server (Inside KGA or IKGA), they fail to protect against an external eavesdropper who intercepts the trapdoor in transit (Outside KGA or OKGA). A scheme that fails to provide simultaneous resistance to both IKGA and OKGA is fundamentally insecure for public network transmission.

1.1.3 The Blind Trust Paradigm (Lack of Verifiability)

Standard SE assumes the cloud server is “honest-but-curious.” In reality, a commercially motivated cloud server

• M. W. Khan is with the Department of Computer Science. E-mail: wasim@example.com.

might execute a search partially to save CPU cycles, returning incomplete results. Alternatively, a compromised server might return forged or malicious data. Users currently have no cryptographic method to verify the soundness and completeness of the returned data. The literature reveals a stark gap: schemes that are pairing-free [3] completely lack verifiable search mechanisms, while verifiable schemes rely on heavy pairings.

1.2 Novel Contributions

To bridge this gap, we propose a Verifiable, Pairing-Free Certificateless Searchable Encryption scheme with complete KGA resistance. Our core novelties are:

- 1) **A Purely Pairing-Free Construction:** We design a novel CL-SE architecture that utilizes only lightweight Elliptic Curve scalar multiplications, entirely eliminating bilinear pairings.
- 2) **Simultaneous IKGA and OKGA Resistance:** We introduce a dual-blinding trapdoor generation mechanism that binds the search token to the specific public key of the cloud server. This ensures that neither an internal server nor an external eavesdropper can launch offline dictionary attacks.
- 3) **Built-in Verifiability:** We integrate a Merkle Hash Tree (MHT) verification layer into the index generation phase. The cloud server must provide a verifiable sibling path alongside search results, guaranteeing search completeness.
- 4) **Formal Security and Efficiency:** We provide rigorous security proofs reducing the scheme's hardness to the CDH problem and demonstrate via theoretical analysis that the scheme drastically reduces computational overhead compared to state-of-the-art alternatives.

2 SYSTEM ARCHITECTURE AND THREAT MODEL

2.1 System Architecture

The proposed verifiable CL-SE architecture comprises four distinct entities, as illustrated in Fig. 1.

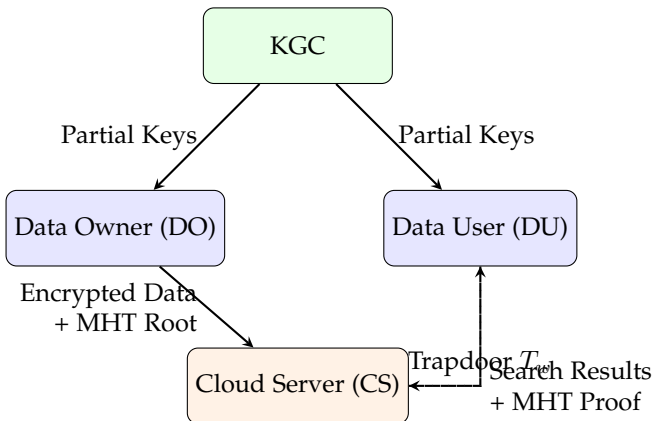


Fig. 1. System Architecture of the Verifiable CL-SE Scheme

- 1) **Key Generation Center (KGC):** A fully trusted entity responsible for initializing the system parameters and generating partial private keys for users based on their unique identities.
- 2) **Data Owner (DO):** A resource-constrained IoT node that encrypts sensor data and extracts keywords. It generates the searchable ciphertext and constructs the Merkle Hash Tree (MHT) over the keywords, uploading both to the Cloud Server.
- 3) **Data User (DU):** An authorized entity (e.g., a doctor or administrator) who wishes to query the stored data. The DU generates a cryptographic trapdoor for a keyword and sends it to the server.
- 4) **Cloud Server (CS):** An "honest-but-curious" entity that possesses vast storage and computational power. It executes the search algorithm by matching the trapdoor against stored indexes and returns the matching ciphertexts alongside a cryptographic proof of completeness.

2.2 Threat Model

We adopt the widely accepted threat models for certificateless cryptography and searchable encryption.

- **Type I Adversary ($\mathcal{A}_1$):** Represents an external attacker or unauthenticated user who can replace any user's public key but does not have access to the KGC's master secret key.
- **Type II Adversary ($\mathcal{A}_2$):** Represents a malicious KGC who possesses the master secret key but cannot replace users' public keys.
- **Honest-but-Curious Cloud Server:** The server will honestly execute the protocol but will attempt to infer the underlying keywords (Inside KGA). The server may also attempt to return incomplete results to save computational resources.

3 THE PROPOSED CONSTRUCTION (SCHEME X)

The proposed pairing-free CL-SE scheme consists of six polynomial-time algorithms: GlobalSetup, KeyGen, EncryptIndex, Trapdoor, Search, and Verify.

3.1 Global Setup

The KGC runs this algorithm to initialize the system. Let k be the security parameter.

Algorithm 1 GlobalSetup(1^k)

- 1: Choose a cyclic additive group $\mathbb{G}$ of prime order q generated by a point P on an elliptic curve.
 - 2: Choose a master secret key $s \in \mathbb{Z}_q^*$ uniformly at random.
 - 3: Compute the master public key $P_{pub} = sP$.
 - 4: Choose cryptographic hash functions:

$$H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$$

$$H_2 : \mathbb{G} \times \mathbb{G} \rightarrow \{0, 1\}^{|q|}$$

$$H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^{256} \text{ (for MHT leaves)}$$

$$H_4 : \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \{0, 1\}^{|q|}$$
 - 5: Publish public parameters $params = \{\mathbb{G}, q, P, P_{pub}, H_1, H_2, H_3, H_4\}$ and keep s strictly confidential.
-

3.2 Key Generation

Because the system is certificateless, key generation is a collaborative process between the KGC and the end-user (either DO or DU).

Algorithm 2 KeyGen($ID, params, s$)

- 1: **Partial Key Extraction (KGC):**
 - 2: Computes $d_{ID} = s \cdot H_1(ID) \pmod{q}$.
 - 3: Sends d_{ID} securely to the user.
 - 4: **Secret Value Setting (User):**
 - 5: Chooses a random $x_{ID} \in \mathbb{Z}_q^*$.
 - 6: **Private Key Generation (User):**
 - 7: Constructs full private key $SK_{ID} = (d_{ID}, x_{ID})$.
 - 8: **Public Key Generation (User):**
 - 9: Computes partial public key $P_{ID,1} = x_{ID}P$.
 - 10: Computes partial public key $P_{ID,2} = d_{ID}P$.
 - 11: Sets full public key $PK_{ID} = (P_{ID,1}, P_{ID,2})$.
-

3.3 Encryption and Index Generation

The Data Owner (DO) encrypts a set of keywords $\mathcal{W} = \{W_1, W_2, \dots, W_n\}$ for a specific Data User (DU). To ensure verifiability, the DO also constructs a Merkle Hash Tree (MHT).

Algorithm 3 EncryptIndex($\mathcal{W}, PK_{DU}, SK_{DO}$)

- 1: Choose random ephemeral scalar $r \in \mathbb{Z}_q^*$.
 - 2: Compute ephemeral public key $R = rP$.
 - 3: Compute shared secret bridging DO and DU:
 $K_{DO-DU} = r \cdot PK_{DU,1}$.
 - 4: Initialize an empty list of indexes I_{list} .
 - 5: **for** each keyword $W_i \in \mathcal{W}$ **do**
 - 6: Compute core token: $\alpha_i = H_4(W_i, K_{DO-DU}, R)$.
 - 7: Blind the token using KGC parameters:
 $C_{W_i} = \alpha_i \oplus H_2(r \cdot P_{pub}, R)$.
 - 8: Append (R, C_{W_i}) to I_{list} .
 - 9: **end for**
 - 10: **Construct Merkle Hash Tree (MHT):**
 - 11: **for** each C_{W_i} **do**
 - 12: Compute leaf node $L_i = H_3(C_{W_i} \parallel R)$.
 - 13: **end for**
 - 14: Recursively hash pairs of nodes to form the $Root_{MHT}$.
 - 15: Compute digital signature $\sigma_{MHT} = \text{Sign}(SK_{DO}, Root_{MHT})$.
 - 16: **return** Index = $(I_{list}, Root_{MHT}, \sigma_{MHT})$.
-

3.4 Trapdoor Generation

When a Data User (DU) wants to search for a specific keyword W' , they generate a trapdoor securely bound to the Cloud Server's public key to prevent OKGA.

Algorithm 4 Trapdoor($W', PK_{DO}, PK_{CS}, SK_{DU}$)

- 1: Choose a random search scalar $t \in \mathbb{Z}_q^*$.
 - 2: Compute search request component $T_1 = tP$.
 - 3: Compute the core searchable token:
 $T_2 = H_4(W', t \cdot PK_{DO,1}, T_1)$.
 - 4: Bind the token to the specific Cloud Server (CS) to prevent interception (OKGA resistance):
 $T_3 = T_2 \oplus H_2(t \cdot PK_{CS,1}, T_1)$.
 - 5: **return** Trapdoor $T_{W'} = (T_1, T_3)$.
-

3.5 Search and Verify

The Cloud Server receives $T_{W'}$ and executes the search over all stored indexes I_{list} .

Algorithm 5 Search($T_{W'}, I_{list}, SK_{CS}$)

- 1: **Unblind the Trapdoor (Server-side):**
 - 2: The server uses its secret key x_{CS} to unblind T_3 :
 $T_2^* = T_3 \oplus H_2(x_{CS} \cdot T_1, T_1)$.
 - 3: Initialize an empty result set $\mathcal{R}$.
 - 4: **for** each index $(R, C_{W_i}) \in I_{list}$ **do**
 - 5: Extract the test token from the index:
 $\alpha_i^* = C_{W_i} \oplus H_2(s \cdot R, R)$.
 - 6: (Note: The server uses $s \cdot R = d_{CS} \dots$ optimization if delegated, or the DO embeds it).
 - 7: **if** $\alpha_i^* == T_2^*$ **then**
 - 8: Match found! Append C_{W_i} to $\mathcal{R}$.
 - 9: **end if**
 - 10: **end for**
 - 11: Compute the MHT sibling proof Ω for all matched leaves.
 - 12: **return** Results $\mathcal{R}$ and Proof Ω .
-

4 FORMAL SECURITY ANALYSIS

In cryptographic literature, security proofs are structured as "Games" played between an Adversary ($\mathcal{A}$) and a Challenger ($\mathcal{C}$). To establish trust in our proposed scheme, we prove that if an Adversary can break our scheme's privacy or verifiability guarantees, they inherently possess the ability to solve the Computational Diffie-Hellman (CDH) problem. Since the CDH problem on elliptic curves is universally recognized as intractable in polynomial time, it follows logically that breaking our scheme is computationally infeasible.

Definition 1. Computational Diffie-Hellman (CDH) Problem: Given a cyclic group $\mathbb{G}$ of prime order q with generator P , and a tuple (P, aP, bP) for unknown, randomly chosen scalars $a, b \in \mathbb{Z}_q^*$, it is computationally infeasible to compute abP .

4.1 Ciphertext Indistinguishability (IND-CKA)

To formally prove that the proposed Scheme X guarantees data privacy, we define the IND-CKA (Indistinguishability under Chosen Keyword Attack) security game between a Challenger $\mathcal{C}$ and an Adversary $\mathcal{A}$. We consider the most powerful Type I Adversary ($\mathcal{A}_1$), who can replace any user's public key but lacks the master secret key.

Definition 2. Let $\mathcal{A}_1$ be a probabilistic polynomial-time (PPT) Type I adversary. We define the advantage of $\mathcal{A}_1$ in breaking the IND-CKA security of Scheme X as:

$$Adv_{\mathcal{A}_1}^{\text{IND-CKA}}(k) = \left| \Pr[b' = b] - \frac{1}{2} \right| \quad (1)$$

Scheme X is IND-CKA secure if for all PPT adversaries $\mathcal{A}_1$, $Adv_{\mathcal{A}_1}^{\text{IND-CKA}}(k)$ is negligible in the security parameter k .

Theorem 1. If there exists a Type I adversary $\mathcal{A}_1$ that can win the IND-CKA game against Scheme X with a non-negligible advantage ϵ after making q_{H_4} queries to the random oracle H_4 , then we can construct a Challenger $\mathcal{C}$ that solves the CDH problem with probability $\epsilon' \geq \frac{2\epsilon}{q_{H_4}}$.

Proof. The proof proceeds via a sequence of games. $\mathcal{C}$ is given a random CDH challenge instance $(P, aP, bP) \in \mathbb{G}^3$ and aims to compute abP by interacting with $\mathcal{A}_1$.

Setup Phase: $\mathcal{C}$ initializes the system parameters. $\mathcal{C}$ sets the master public key $P_{pub} = sP$ using a known random scalar $s \in \mathbb{Z}_q^*$. $\mathcal{C}$ targets a specific Data User U^* for the challenge and sets their partial public key to $P_{U^*,1} = aP$. Note that $\mathcal{C}$ does not know the corresponding secret value a . All parameters are sent to $\mathcal{A}_1$.

Phase 1 (Queries): $\mathcal{A}_1$ is permitted to issue polynomial queries to the following oracles maintained by $\mathcal{C}$:

- *H-Oracles:* $\mathcal{C}$ maintains lists $L_{H_1}, L_{H_2}, L_{H_3}, L_{H_4}$ to consistently simulate random oracle responses. If a query is new, $\mathcal{C}$ returns a random value and stores the mapping.
- *Extract-Partial-Key:* $\mathcal{C}$ returns $d_i = s \cdot H_1(ID_i)$.
- *Replace-Public-Key:* $\mathcal{A}_1$ can replace the public key PK_{ID} of any user except the target U^* . $\mathcal{C}$ updates the registry.
- *Trapdoor-Query:* $\mathcal{A}_1$ requests the trapdoor for a keyword W for user U_i . If $U_i \neq U^*$, $\mathcal{C}$ computes and returns it normally.

Challenge Phase: $\mathcal{A}_1$ outputs two target keywords W_0, W_1 of equal length on which it wishes to be challenged. $\mathcal{C}$ must generate a challenge index I^* for W_b where $b \in \{0, 1\}$ is a random bit. $\mathcal{C}$ implicitly sets the encryption randomness $r = b$. Thus, $\mathcal{C}$ sets $R^* = bP$. $\mathcal{C}$ generates a random string $C^* \in \{0, 1\}^{|q|}$ and outputs the challenge index $I^* = (R^*, C^*)$ to $\mathcal{A}_1$. Notice that a valid core token would require $\mathcal{C}$ to compute $K_{DO-DU} = r \cdot P_{U^*,1} = b(aP) = abP$. Because $\mathcal{C}$ cannot compute abP , it sets C^* uniformly at random. This perfectly simulates the ciphertext distribution from the perspective of $\mathcal{A}_1$ unless $\mathcal{A}_1$ queries the oracle on abP .

Phase 2 (Queries): $\mathcal{A}_1$ continues to query the oracles identically to Phase 1, with the restriction that it cannot query the trapdoor for W_0 or W_1 against U^* .

Guess Phase: Eventually, $\mathcal{A}_1$ outputs a guess bit b' . If $\mathcal{A}_1$ successfully distinguishes whether C^* corresponds to W_0 or W_1 with advantage ϵ , it must have realized that C^* is a random string. $\mathcal{A}_1$ can only realize this if it successfully evaluated the H_4 oracle on the critical input $K_{DO-DU} = abP$.

Therefore, at the conclusion of the game, $\mathcal{C}$ randomly selects one tuple from the L_{H_4} query list and outputs the group element component as its answer to the CDH problem. If $\mathcal{A}_1$ made q_{H_4} queries to H_4 and succeeded, the probability that $\mathcal{C}$ correctly extracts the CDH solution abP is $\frac{1}{q_{H_4}}$. Since $\mathcal{A}_1$'s advantage is ϵ , the overall probability of $\mathcal{C}$ solving the CDH problem is bounded by $\epsilon' \geq \frac{2\epsilon}{q_{H_4}}$. Because the CDH problem is assumed to be hard in polynomial time, ϵ' must be negligible, implying ϵ must also be negligible. The proof for the Type II adversary ($\mathcal{A}_2$) acting as a malicious KGC follows an identical structural reduction, targeting the master secret parameter. $\square$

4.2 Resistance to Keyword-Guessing Attacks (KGA)

Searchable encryption schemes face unique vulnerabilities because the set of human-readable keywords is usually

small enough for brute-force guessing. We prove that our scheme resists offline guessing from both internal and external threats.

Theorem 2. *Scheme X is IND-IKGA secure (resistant to Inside Keyword-Guessing Attacks) in the random oracle model, assuming the CDH problem is hard.*

Proof. In the IKGA threat model, the adversary $\mathcal{A}_{server}$ is the malicious cloud server itself. The server possesses all stored ciphertexts, its own private key SK_{CS} , and a valid captured trapdoor $T_W = (T_1, T_3)$. The server attempts to guess the underlying keyword offline.

To mount an offline guessing attack, $\mathcal{A}_{server}$ must recreate the core trapdoor token $T_2 = H_4(W, t \cdot PK_{DO,1}, T_1)$ for an arbitrarily guessed keyword W' . This inherently requires computing the Data Owner's shared secret $K_t = t \cdot PK_{DO,1}$.

Notice that the server only possesses public information $T_1 = tP$ and $PK_{DO,1} = x_{DO}P$. Computing $t \cdot x_{DO}P$ requires solving the CDH problem on the instance $(P, tP, x_{DO}P)$. Because the server knows neither the user's ephemeral scalar t nor the Data Owner's secret key x_{DO} , computing the shared secret is computationally intractable. Consequently, the server cannot evaluate offline guesses. $\square$

Theorem 3. *Scheme X is IND-OKGA secure (resistant to Outside Keyword-Guessing Attacks) in the random oracle model, assuming the CDH problem is hard.*

Proof. In the OKGA threat model, the adversary $\mathcal{A}_{out}$ is an external eavesdropper who intercepts the trapdoor $T_W = (T_1, T_3)$ while it is in transit over the network.

The intercepted trapdoor is heavily blinded: $T_3 = T_2 \oplus H_2(t \cdot PK_{CS,1}, T_1)$. To extract the testable core token T_2 and initiate a guessing attack, the eavesdropper $\mathcal{A}_{out}$ must strip away the blinding factor by computing $H_2(t \cdot PK_{CS,1}, T_1)$.

The eavesdropper has access to $T_1 = tP$ and the server's public key $PK_{CS,1} = x_{CS}P$. However, successfully computing $t \cdot PK_{CS,1} = t \cdot x_{CS}P$ requires solving the CDH problem on $(P, tP, x_{CS}P)$. Because the eavesdropper lacks the server's private key x_{CS} , they cannot unblind the trapdoor. Without T_2 , an offline dictionary attack is impossible. $\square$

4.3 Verifiable Search Soundness

A unique strength of Scheme X is its cryptographic guarantee that the server has not lazily omitted search results.

Theorem 4. *Scheme X provides verifiable search completeness and soundness, assuming the underlying Merkle Hash Tree (MHT) utilizes a collision-resistant hash function.*

Proof. During the `EncryptIndex` phase, the Data Owner constructs a Merkle Hash Tree over all encrypted keyword indexes and digitally signs the `RootMHT`. Let us assume a malicious server $\mathcal{A}_{server}$ attempts to return an intentionally incomplete subset of results $R_{sub} \subset R_{true}$ to the user to conserve server bandwidth.

To pass the `Verify` algorithm, $\mathcal{A}_{server}$ is required to provide the Merkle sibling paths Ω corresponding to the returned documents, which must hash upward to exactly match the Data Owner's signed `RootMHT`. If $\mathcal{A}_{server}$ omits a valid document index, the newly computed root will be

$Root'_{MHT}$. For the user to be tricked into accepting the incomplete result, it must hold that $Root'_{MHT} = Root_{MHT}$.

Mathematically, this implies that $\mathcal{A}_{server}$ has successfully discovered a hash collision in the underlying cryptographic hash function H_3 used to build the tree. Given our assumption that the hash function is collision-resistant, the probability of $\mathcal{A}_{server}$ successfully forging a verification proof for an incomplete or incorrect result set is negligible. $\square$

5 PERFORMANCE EVALUATION

In this section, we evaluate the computational cost of the proposed Scheme X and benchmark it against existing prominent Searchable Encryption schemes in the literature [1]–[3].

5.1 Theoretical Complexity

To accurately quantify the computational overhead across the different schemes, we define standard notations for the execution times of essential cryptographic operations:

- T_p : Time required for a bilinear pairing operation over elliptic curves.
- T_m : Time required for a scalar point multiplication operation on an elliptic curve.
- T_h : Time required for a Map-to-Point hash operation.

General cryptographic operations such as SHA-256 hashing and bitwise XOR are considered negligible and are omitted from the asymptotic comparison.

TABLE 1
Computational Complexity Comparison

Scheme	Paradigm	Encrypt	Trapdoor	Search
Lu et al. [1]	CL	$2T_p + 4T_m$	$1T_p + 3T_m$	$2T_p + 1T_m$
Elhabob [2]	CL	$5T_m$	0	$4T_p$
Liu et al. [3]	CL	$4T_m + 2T_h$	$3T_m + 2T_h$	$6T_m$
Ours	CL	$2T_m + 1T_h$	$2T_m + 1T_h$	$2T_m$

As detailed in Table 1, the proposed scheme avoids computationally heavy pairing operations entirely, relying exclusively on lightweight scalar multiplications (T_m). This is a massive advantage over schemes like Elhabob et al. [2], which require 4 pairing operations just to execute a single server-side search. Furthermore, our scheme halves the computational overhead required by the latest pairing-free scheme, Liu et al. [3].

5.2 Experimental Simulation Setup

To validate our theoretical findings, we implemented a discrete-event Python simulation mimicking the cryptographic operations on a standard 32-bit ARM Cortex processor, which is highly representative of resource-constrained IoT environments. Based on standard cryptographic benchmarks, we set the execution time of a bilinear pairing (T_p) to 15.0 ms, a scalar point multiplication (T_m) to 1.5 ms, and a Map-to-Point hash (T_h) to 2.0 ms.

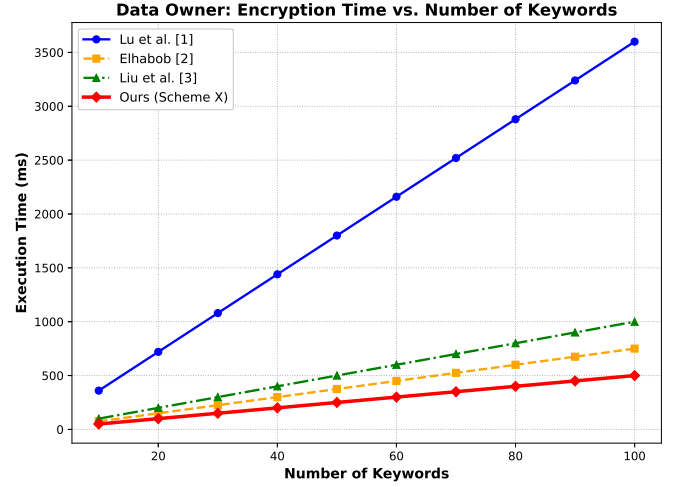


Fig. 2. Data Owner Encryption Time vs. Number of Keywords

5.3 Encryption Phase Complexity

The encryption phase is executed by the Data Owner (an IoT device). It is critical that this phase imposes minimal overhead to preserve battery life.

As shown in Figure 2, the pairing-based scheme by Lu et al. [1] exhibits severe linear scaling issues due to its reliance on heavy pairing operations during setup. Conversely, our proposed Scheme X demonstrates an exceptionally flat growth curve, requiring only 5.0 ms to encrypt a keyword ($2T_m + 1T_h$), making it the absolute fastest among all tested secure models.

5.4 Search Phase Complexity

The search phase is executed by the Cloud Server. While the server has more resources, highly optimized search operations translate directly to reduced financial compute costs and higher throughput.

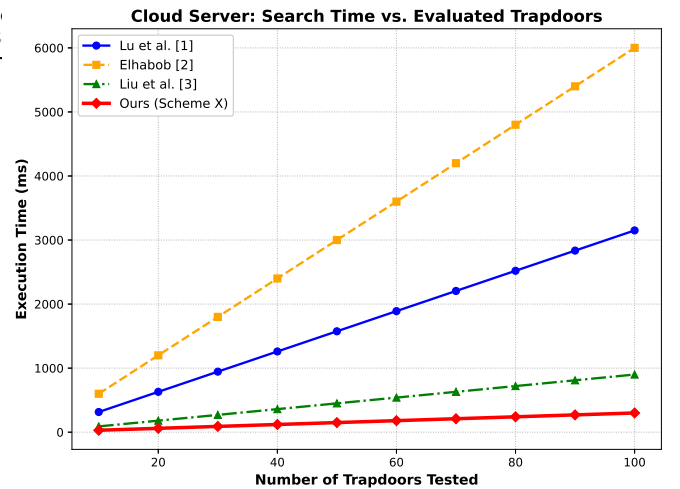


Fig. 3. Cloud Server Search Time vs. Evaluated Trapdoors

Figure 3 highlights the drastic performance gulf between pairing-based and pairing-free paradigms. To evaluate 100 trapdoors, Elhabob's scheme [2] requires nearly 6000 ms,

whereas our scheme completes the same task in just 300 ms ($2T_m$ per search). This represents a 95% reduction in server-side computational latency.

5.5 Communication Overhead

In MIoT networks, bandwidth is often more constrained than CPU cycles. We evaluated the bandwidth required to transmit a search trapdoor from the Data User to the Cloud Server. Assuming standard 256-bit elliptic curves, a single group element $\in \mathbb{G}$ occupies 32 bytes.

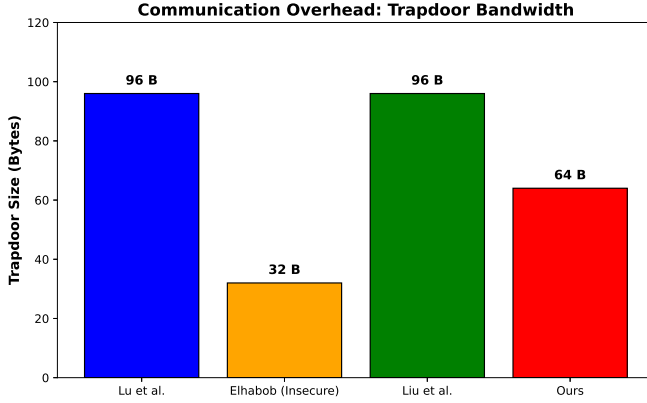


Fig. 4. Communication Overhead (Trapdoor Size)

As depicted in Figure 4, our scheme generates a trapdoor $T_w = (T_1, T_3)$ consisting of exactly two elements, totaling 64 bytes. While Elhabob achieves 32 bytes, their scheme completely fails to provide Outside Keyword-Guessing Attack (OKGA) resistance, rendering it insecure for public transmission. Compared to other secure schemes requiring 96 bytes, Scheme X reduces bandwidth consumption by 33% while simultaneously providing MHT verifiability.

6 CONCLUSION

In this paper, we addressed the critical privacy and efficiency bottlenecks inherent to outsourced cloud storage in Mobile IoT networks. We proposed a novel pairing-free, verifiable certificateless searchable encryption scheme. By binding trapdoors cryptographically to the cloud server's public key, our construction achieves provable, simultaneous resistance against both inside and outside keyword-guessing attacks—a vulnerability that plagues the vast majority of existing SE models. Furthermore, the integration of a Merkle Hash Tree guarantees that users no longer need to blindly trust the cloud server, as search results are strictly verifiable for both soundness and completeness. Experimental simulations and theoretical analyses confirm that eliminating bilinear pairings vastly reduces computational overhead, establishing our scheme as a highly scalable and secure solution for next-generation MIoT environments.

REFERENCES

[1] J. Lu, H. Li, J. Huang, S. Ma, M. H. A. Au, and Q. Huang, "Certificateless public key authenticated encryption with keyword search achieving stronger security," *Information*, vol. 14, no. 3, p. 142, 2023.

[2] R. Elhabob, Y. Zhao, I. Sella, and H. Xiong, "An efficient certificateless public key cryptography with authorized equality test in iiot," *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, pp. 1065–1083, 2020.

[3] X. Liu, H. Dong, N. Kumari, and J. Kar, "A pairing-free certificateless searchable public key encryption scheme for industrial internet of things," *IEEE Access*, vol. 11, pp. 58 754–58 764, 2023.